HIERO

A persistent cognitive substrate is only as useful as it is decodable. JARVIS depends on hundreds of memory files written in a format I call HIERO — operator-density prose with project-specific shorthand, structured for fast parse and high density. The format works. The format also doesn't have a published dictionary. Anyone wanting to adopt the architecture, contribute to the corpus, or extend the methodology hits a wall the moment they encounter an actual HIERO file: standard math operators they recognize, project-specific glyphs they don't, conventions they have to infer from reading enough of the corpus to triangulate.

This paper is the dictionary. It also makes the broader argument: a format without a dictionary is a personal compression scheme. A format with a dictionary is a protocol. The difference is whether the architecture survives its original operator, can be adopted by other operators, and earns the substrate-port property the rest of the JARVIS body of work has demonstrated for other substrates. HIERO has been the load-bearing format under JARVIS for a year. Without this dictionary, it has been load-bearing but not decodable. With it, the format can be used by anyone willing to write under its discipline.

This paper exists for two reasons. The first is operational — anyone, including future agents and future versions of the original operator, needs the dictionary to read the corpus. The second is structural — publishing the dictionary is the move that converts JARVIS from personal architecture to public protocol. That conversion is the capstone of the entire body of work. Substrate-port from one operator to many is the same move the methodology has been performing across substrates, applied finally to itself.

Why HIERO exists

Memory writes face a specific failure mode I've seen recur across long sessions: prose that reads cleanly under attention degrades into noise under repeated load. A primitive memory file gets written in clean prose, gets loaded into context a thousand times across sessions, and the prose-parse cost compounds linearly while the information content stays fixed. Eventually the cost exceeds the benefit and the file becomes effectively non-loaded — the model skims past it because the parse-time-to-information ratio is wrong.

The fix is to compress the format such that the parse cost falls and the information density rises simultaneously. Standard prose loses to dense logical notation on this trade. A primitive that says "for any session involving partner-facing claims, verify credentials in profile memory before drafting any text" can be expressed as `∀ partner-facing claims ⇒ grep profile-memory FIRST` with no information loss and roughly an order of magnitude less parse cost.

The HIERO format formalizes that compression. Every memory file targets: - Operator density above 0.005 (one operator per 200 characters) - Average line length under 120 characters - Multi-sentence lines under three per file - Frontmatter structure that makes file purpose machine-extractable

A pre-write hook enforces these targets. Files that fail hook validation get rejected; the operator (me, in this case) has to recompress before the write succeeds. The hook has refused my own writes routinely. The discipline is real because it's enforced, not because it's aspirational.

What HIERO is not

A few clarifications before the dictionary, because misunderstanding the format leads to wrong critiques.

HIERO is not minified code. The compression is in the symbol layer, not in whitespace removal or identifier shortening. Lines are short for parsing reasons, not space reasons; symbols are dense for information reasons, not aesthetic reasons.

HIERO is not pseudo-mathematical formalism for its own sake. Every symbol used in the format has a specific meaning that does the work of an English phrase. `⇒` is “implies / triggers / produces” — it replaces a connective phrase that would otherwise consume bytes and parse cycles. `∀` is “for all” — it replaces “every time / whenever / in every case where.” The compression is functional, not decorative.

HIERO is not exclusionary. It looks intimidating to readers who haven’t seen the dictionary, which is precisely the problem this paper solves. Once decoded, HIERO is more readable than the equivalent prose because the symbols carry meaning faster than English connectives do.

HIERO is not lossless on every dimension. It loses some warmth, some narrative texture, some of the rhetorical features that make prose persuasive. The trade is intentional — memory files don’t need to persuade, they need to inform a model that’s about to act. Persuasion belongs in papers like this one, not in primitive files.

The dictionary

The format has three layers of vocabulary. Universal operators, drawn from standard logic and mathematics, work without project-specific knowledge. Project-specific prefixes and section markers identify file types and content categories. Recurring idioms encode common patterns of reasoning that come up across many primitives.

Layer 1: Universal operators

These have been standardized in mathematical and logical notation for decades. Anyone with a college-level math background recognizes them; the dictionary entries below specify how they’re used in HIERO context.

Symbol	Meaning	Example use
`∀`	For all / every / whenever	`∀ public draft ⇒ verify creds first`
`∃`	There exists / some	`∃ counter-example ⇒ rule needs scoping`
`⇒`	Implies / triggers / produces	`pattern × 3+ ⇒ surface as primitive`
`⇔`	If and only if / iff	`valid ⇔ all gates pass`
`∧`	And / both required	`∀ commit ⇒ tests-pass ∧ review-done`
`∨`	Or / at least one	`signal: "draft" ∨ "ready to paste"`
`¬`	Not / forbidden / negation	`¬ blockquote in pasted drafts`
`⊥`	Forbidden / blocked / impossible	`local FS path in public doc ⊥`
`⊤`	True / always permitted	`read-only access ⊤`
`→`	Transitions to / leads to	`PENDING → SETTLED`
`↦`	Maps to / corresponds to	`commit reveal ↦ MEV elimination`
`✓`	Yes / valid / passes	`✓ GH URL preferred`
`✗`	No / invalid / fails	`✗ "founder of VibeSwap"`
`∅`	Null / prohibited / explicitly empty	`∅ "internal-only" tag (rare)`

Symbol	Meaning	Example use
`↑` / `↓`	Increase / decrease	`attention ↓ over session length`
`∈` / `∉`	Member of / not member	`Will ∈ trust-default cohort`
`⊆` / `⊂`	Subset / proper subset	`audit-cycles ⊆ TRP-loops`
`×`	Multiplication / combination	`density × stability × match-speed`
`≥` `≤` `≠` `≡` `≈`	Comparison operators	`density ≥ 0.005`, `intent ≡ outcome`

Layer 2: Project-specific prefixes

JARVIS files use single-character prefixes followed by `·` to identify file type. The prefix matters because it conditions how the file gets loaded — different file types have different load triggers and different render contexts.

Prefix	File type	Filename pattern
`P·`	Primitive	`primitive_<name>.md`
`F·`	Feedback rule	`feedback_<name>.md`
`J·`	Project memory	`project_<name>.md`
`U·`	User-context file	`user_<name>.md`
`R·`	Reference pointer	`reference_<name>.md`
`O·`	Protocol document	`protocol_<name>.md`
`M·`	Generic memory (no prefix in filename)	`<name>.md`

Cross-references between files use the `[Name](type·slug)` pattern. Resolved by the audit script; checked for orphans on every memory write; closure is what produces the 100% DE-score the architecture maintains.

Layer 3: Section markers and structural conventions

Memory files have predictable section structure that lets the model parse them quickly. Some sections are universal across file types; others are file-type specific.

Universal sections (any memory file): - `**Rule**:` — the actual principle stated as compactly as possible - `**Why**:` — origin or motivation, often with a block-quoted Will exact-words anchor - `**How**: / **How to apply**:` — operational guidance - `**Connected**:` — cross-references to related primitives - `**Trigger**:` — when this primitive should fire

Section taxonomy in MEMORY.md: - `[PRE-FLIGHT]` — load before any work; violations are irreversible - `[BOOT]` — identity, paths, baseline orientation - `[META-PRINCIPLE]` — load-bearing above all situation rules - `[POST-HOC:HOT]` — always-applicable situation rules - `[POST-HOC:WARM-MAP]` — load matching warm-map file on situation - `[ACTIVE]` — current posture, always relevant - `[PING]` — always-on notification rules - `[TOKENOMICS]` — load-bearing monetary framing

Recurring small-caps section markers (visual structuring within longer files): - `↻TRP` — Trust-Rebuilding Protocol primitives - `↻INT` — Integration primitives - `↻GOV` — Governance primitives - `↻REV` — Review / RSI primitives - `↻METH` — Methodology / theory layer - `↻SELF` — Self-improvement / meta architecture

The pattern is `↻<3-letter category in small caps>`. New categories follow the same shape.

Frontmatter convention (every memory file):

```
---
name: <CamelCaseName>
description: <one-line, used for relevance matching during recall>
```

```
type: <primitive | feedback | project | user | reference | protocol>
---
```

The `description` field is the index entry — it's what the auto-loader matches against current-conversation context to decide whether to surface this file. Description quality determines load accuracy.

Layer 4: Recurring idioms

Patterns that come up across many primitives. These aren't single symbols but combinations that encode common reasoning structures.

Idiom	Meaning
`pattern × N+ ⇒ surface candidate`	Counting-threshold rule; pattern repeated N or more times triggers crystallization
`∀ X ⇒ check Y FIRST`	Precondition rule; X cannot proceed without Y
`path-X ⊥ ⇒ replace w/ Y`	Forbid-and-replace; X is forbidden; valid alternative is Y
`Apply @ X ¬ Y`	Application gating; the rule fires at point X, not point Y
`signal: "phrase"`	Detection cue; specific phrase triggers the primitive
`[N·name](path)`	Cross-reference to another primitive
`> *"..."*`	Block-quoted Will exact words; serves as authority anchor
`✓ ① / ② / ③`	Numbered options listed in priority order
`⇉`	Composition of mechanisms (when distinguished from single ⇒)

These idioms accrete over time as recurring patterns in the corpus. The canonical list above captures the common ones as of the dictionary's first publication; new idioms get added when patterns repeat enough to warrant compression.

A worked example

Below is an actual memory file, fully annotated against the dictionary. The example demonstrates what reading HIERO with the dictionary in hand looks like.

File: `primitive_substrate-geometry-match.md` (excerpt)

```
---
name: SubstrateGeomMatch
description: Hermetic-maxim @ mechanism. Macro={fractal,power-law} ⇒
micro={Fibonacci,golden-ratio}.
type: primitive
---

**[SubstrateGeomMatch](P·substrate-geometry-match)** — hermetic-maxim @
mechanism.
macro={fractal,power-law} ⇒ micro={Fibonacci,golden-ratio}.
mismatch=failure-mode, generator(First-Available-Trap).
WHAT-shape.
```

Decoded: - Frontmatter: file is a primitive named `SubstrateGeomMatch`, with description "Hermetic-maxim at the mechanism level. When the macro substrate has form {fractal, power-law}, the micro mechanism should match with form {Fibonacci, golden-ratio}." - Rule line: References the primitive's own slug (`P·substrate-geometry-match`) — self-referential identification. "Hermetic-maxim @ mechanism" means the hermetic principle "as above, so below" applied at mechanism design. - Compressed body: macro form (in set notation: fractal or power-law) implies (⇒) micro form (Fibonacci or golden-ratio). When mismatched,

this is the failure-mode that generates the first-available-trap pattern. - Tag: `WHAT-shape` — categorizes this primitive as addressing the WHAT question (versus HOW or WHY) of mechanism design.

Reading this without the dictionary requires either deep prior context with the body of work or substantial inference from the surrounding corpus. Reading it with the dictionary takes seconds and produces no information loss.

Discipline of dictionary maintenance

A dictionary that doesn't get maintained drifts away from the corpus it claims to describe. Three rules govern dictionary maintenance:

New idioms get added when they appear three or more times in the corpus. The same threshold that promotes patterns to primitives. Below three instances, an idiom is just a one-off compression that may not generalize. At three or more, it has earned canonical status.

Removed conventions get marked deprecated, not deleted. The corpus may still contain old conventions; readers parsing old files need the deprecated entries to decode them. Deletion would create silent comprehension failures on historical files.

The dictionary lives where the corpus lives. This file ships in the same repository as the corpus it describes. If the corpus moves, the dictionary moves with it. Co-location prevents the dictionary from going stale relative to its corpus.

The audit script that produces the 100% DE-score also flags any glyph or convention used in the corpus that isn't in the dictionary. New conventions that pass the three-instances threshold get surfaced for inclusion; the dictionary stays in sync with actual usage rather than aspirational specification.

Why this is the capstone

The JARVIS body of work has demonstrated the substrate-port methodology across roughly forty substrates outside DeFi. The argument has been that the methodology generalizes — same pattern produces useful systems across substrates that share neither participants nor failure modes nor implementation languages.

The strongest test of that argument is whether JARVIS itself can be ported. Can the architecture I've been running on Claude be adopted by another operator on another substrate? Can the discipline scale beyond one person's cognitive substrate?

The answer is no without the HIERO dictionary. The persistence layer is the architecture's load-bearing core; the persistence layer is written entirely in HIERO; HIERO without a dictionary is decodable only to its original operator. The architecture, however well-designed, has been operator-locked.

With the dictionary, the architecture becomes substrate-portable in the same sense the methodology has demonstrated for everything else. Other operators can read the existing corpus. Other operators can write new primitives that compose with existing ones. Future versions of the original operator can read the corpus without relying on remembered conventions. The Cincinnatus walkaway test — can the system continue operating without its original operator — becomes structurally satisfiable rather than aspirational.

This is the move that closes the recursion. Substrate-port from EVM to AI substrate produced JARVIS. Substrate-port from one operator to many requires HIERO decodable. The dictionary is what makes the second port possible. Once the dictionary is published, the architecture has demonstrated portability across both substrate-changes and operator-changes. The methodology generalizes at both axes.

Closing

The dictionary above is the first public version. It captures the operators, prefixes, sections, and idioms used in the current JARVIS corpus. It will need maintenance — new idioms will appear, old ones may need clarification, the corpus will grow. The discipline of maintenance is itself a primitive (`P·dictionary-maintenance` , presumably written in HIERO once this paper is published).

For anyone reading this who wants to adopt the architecture: clone the JARVIS scaffold, read the dictionary, write your first primitive in HIERO, run the audit script, watch your DE-score. The architecture works. The format compresses without loss. The discipline holds. None of it depends on me anymore.

That last sentence is what the dictionary makes true. It hasn't been true before today.

The format was always a protocol. The dictionary is what publishes the protocol. The architecture was always portable. The dictionary is what proves it.